

```
# Part-of-Speech Tagging using Hidden Markov Models
```

```
# This notebook implements a Part-of-Speech (POS) tagger using a Hidden Markov Model (HMM) and the Viterbi algorithm.
```

```
# The goal of this assignment is to understand how probabilistic sequence models can be used to assign grammatical categories such as
```

```
# Two taggers are implemented:
```

```
# 1. A vanilla Hidden Markov Model POS tagger using the Viterbi algorithm.
```

```
# 2. An improved tagger that handles unknown words using an UNK probability model.
```

```
# The NLTK Treebank corpus with the Universal Tagset is used. All data processing, training, and evaluation are performed inside this
```

```
## Dataset Description
```

```
# The Treebank corpus from the NLTK library is used in this assignment. The corpus contains sentences where each word is annotated with
```

```
# The Universal Tagset is used. This tagset contains a small number of coarse grammatical categories such as: NOUN, VERB, ADJ, ADV, DE
```

```
# Using a smaller tagset simplifies the modeling process while still preserving linguistic structure.
```

```
# The dataset is downloaded programmatically as required.
```

```
import nltk
import random
import math
import pandas as pd
```

```
nltk.download('treebank', quiet=True)
nltk.download('universal_tagset', quiet=True)
```

```
from nltk.corpus import treebank
```

```
## Data Preparation
```

```
# The dataset is loaded using the Universal Tagset.
```

```
# Each sentence consists of a sequence of (word, tag) pairs.  
  
# The dataset is split into: Training set (90%) and Validation set (10%)  
  
# The split is done randomly but with a fixed seed so that results are reproducible.
```

```
sentences = list(treebank.tagged_sents(tagset='universal'))  
  
print("Total sentences:",len(sentences))  
  
print("\nExample sentence:")  
print(sentences[0])
```

Total sentences: 3914

Example sentence:

[('Pierre', 'NOUN'), ('Vinken', 'NOUN'), (',', '.'), ('61', 'NUM'), ('years', 'NOUN'), ('old', 'ADJ'), (',', '.'), ('will', 'VERB'), ('

```
random.seed(42)  
  
random.shuffle(sentences)  
  
split=int(0.9*len(sentences))  
  
train_data=sentences[:split]  
test_data=sentences[split:]  
  
print("Training sentences:",len(train_data))  
print("Validation sentences:",len(test_data))
```

Training sentences: 3522

Validation sentences: 392

```
## Hidden Markov Model Parameter Estimation  
  
## The following probabilities are estimated from the training data.  
  
### Transition Probabilities  
  
## These probabilities represent:
```

```
# P(tag_i | tag_{i-1})

# which is the probability of moving from one tag to the next tag.

# A special START state is used to represent the beginning of a sentence.

### Emission Probabilities

# These probabilities represent:

# P(word | tag)

# which is the probability that a tag produces a given word.

### Laplace Smoothing

# Laplace smoothing is applied to emission probabilities:

#  $P(\text{word}|\text{tag}) = \frac{\text{count}(\text{word},\text{tag})+1}{\text{count}(\text{tag})+V}$ 

# where V is the vocabulary size.

# This prevents zero probabilities.
```

```
tag_counts={}
word_tag_counts={}
transition_counts={}

for sentence in train_data:

    prev_tag="START"

    for word,tag in sentence:

        tag_counts[tag]=tag_counts.get(tag,0)+1

        if tag not in word_tag_counts:
            word_tag_counts[tag]={}

        word_tag_counts[tag][word]=word_tag_counts[tag].get(word,0)+1
```

```
if prev_tag not in transition_counts:
    transition_counts[prev_tag]={}

transition_counts[prev_tag][tag]=transition_counts[prev_tag].get(tag,0)+1

prev_tag=tag
```

```
tags=list(tag_counts.keys())

vocab=set()

for tag in word_tag_counts:
    vocab.update(word_tag_counts[tag].keys())

V=len(vocab)
T=len(tags)

transition_prob={}

for prev_tag in transition_counts:

    transition_prob[prev_tag]={}

    total=sum(transition_counts[prev_tag].values())

    for tag in tags:

        count=transition_counts[prev_tag].get(tag,0)

        transition_prob[prev_tag][tag]=(count+1)/(total+T)

emission_prob={}

for tag in tags:

    emission_prob[tag]={}

    total=tag_counts[tag]

    for word in vocab:
```

```
count=word_tag_counts[tag].get(word,0)

emission_prob[tag][word]=(count+1)/(total+V)
```

```
## Viterbi Algorithm
```

```
# The Viterbi algorithm is used to find the most probable sequence of tags for a sentence.
```

```
# The algorithm uses dynamic programming.
```

```
# For each word in the sentence, the probability of each possible tag is computed based on: Transition probabilities and Emission proba
```

```
# Log probabilities are used to avoid numerical underflow.
```

```
def viterbi_vanilla(sentence):
```

```
    Viterbi=[{}]
```

```
    path={}
```

```
    for tag in tags:
```

```
        word=sentence[0]
```

```
        emit=emission_prob[tag].get(word,1e-6)
```

```
        trans=transition_prob["START"].get(tag,1e-6)
```

```
        Viterbi[0][tag]=math.log(trans)+math.log(emit)
```

```
        path[tag]=[tag]
```

```
    for t in range(1,len(sentence)):
```

```
        Viterbi.append({})
```

```
        new_path={}
```

```
        word=sentence[t]
```

```
        for tag in tags:
```

```
emit=emission_prob[tag].get(word,1e-6)

best_prob=None
best_prev=None

for prev_tag in tags:

    trans=transition_prob[prev_tag].get(tag,1e-6)

    prob=Viterbi[t-1][prev_tag]+math.log(trans)+math.log(emit)

    if best_prob is None or prob>best_prob:

        best_prob=prob
        best_prev=prev_tag

Viterbi[t][tag]=best_prob
new_path[tag]=path[best_prev]+[tag]

path=new_path

best_tag=max(Viterbi[-1],key=Viterbi[-1].get)

return path[best_tag]
```

Handling Unknown Words

```
# Words that do not appear in the training data are called unknown words.

# The vanilla Viterbi algorithm assigns extremely small probabilities to such words.

# To improve performance, an UNK model is introduced.

# For unknown words, emission probabilities are replaced with tag prior probabilities:

#  $P(\text{tag}) = \text{count}(\text{tag}) / \text{total tags}$ 

# This allows the model to assign reasonable tags even for unseen words.
```

```
total_tags=sum(tag_counts.values())

tag_prior={tag:tag_counts[tag]/total_tags for tag in tags}

def viterbi_unk(sentence):

    Viterbi=[{}]
    path={}

    for tag in tags:

        word=sentence[0]

        if word in vocab:
            emit=emission_prob[tag][word]
        else:
            emit=tag_prior[tag]

        trans=transition_prob["START"].get(tag,1e-6)

        Viterbi[0][tag]=math.log(trans)+math.log(emit)

        path[tag]=[tag]

    for t in range(1,len(sentence)):

        Viterbi.append({})
        new_path={}

        word=sentence[t]

        for tag in tags:

            if word in vocab:
                emit=emission_prob[tag][word]
            else:
                emit=tag_prior[tag]

            best_prob=None
            best_prev=None

            for prev_tag in tags:
```

```
trans=transition_prob[prev_tag].get(tag,1e-6)

prob=Viterbi[t-1][prev_tag]+math.log(trans)+math.log(emit)

if best_prob is None or prob>best_prob:

    best_prob=prob
    best_prev=prev_tag

Viterbi[t][tag]=best_prob
new_path[tag]=path[best_prev]+[tag]

path=new_path

best_tag=max(Viterbi[-1],key=Viterbi[-1].get)

return path[best_tag]
```

Evaluation

Accuracy is computed as:

Accuracy = Correctly tagged words / Total words

Two models are evaluated: Vanilla Viterbi and Viterbi with UNK handling

```
def accuracy(dataset,model):
```

```
    correct=0
    total=0
```

```
    for sentence in dataset:
```

```
        words=[w for w,t in sentence]
        true_tags=[t for w,t in sentence]
```

```
        pred_tags=model(words)
```

```
        for p,t in zip(pred_tags,true_tags):
```

```

        total+=1

        if p==t:
            correct+=1

    return correct/total

```

```

vanilla_acc=accuracy(test_data,viterbi_vanilla)

unk_acc=accuracy(test_data,viterbi_unk)

results=pd.DataFrame({

    "Model":["Vanilla Viterbi","Viterbi with UNK"],

    "Accuracy":[vanilla_acc,unk_acc]

})

results

```

	Model	Accuracy	
0	Vanilla Viterbi	0.900850	
1	Viterbi with UNK	0.904552	

Next steps:

[Generate code with results](#)[New interactive sheet](#)

```
## Quantitative Analysis
```

```
# The results show that the Viterbi model with unknown word handling performs better than the vanilla model.
```

```
# The UNK model assigns reasonable probabilities to unseen words which improves tagging accuracy.
```

```
count=0
```

```
for sentence in test_data:
```

```

words=[w for w,t in sentence]
true_tags=[t for w,t in sentence]

v1=viterbi_vanilla(words)
v2=viterbi_unk(words)

if v1!=true_tags and v2==true_tags:

    print("\nSentence:",words)

    print("True:",true_tags)

    print("Vanilla:",v1)

    print("UNK:",v2)

    count+=1

    if count==3:
        break

```

```

Sentence: ['Mr.', 'Hahn', 'also', 'has', 'engineered', 'a', 'surprising', 'turnaround', 'of', 'Georgia-Pacific', '.']
True: ['NOUN', 'NOUN', 'ADV', 'VERB', 'VERB', 'DET', 'ADJ', 'NOUN', 'ADP', 'NOUN', '.']
Vanilla: ['NOUN', 'NOUN', 'ADV', 'VERB', 'ADP', 'DET', 'ADJ', 'NOUN', 'ADP', 'NOUN', '.']
UNK: ['NOUN', 'NOUN', 'ADV', 'VERB', 'VERB', 'DET', 'ADJ', 'NOUN', 'ADP', 'NOUN', '.']

```

```

Sentence: ['The', 'company', 'is', 'contesting', 'the', 'fine', '.']
True: ['DET', 'NOUN', 'VERB', 'VERB', 'DET', 'NOUN', '.']
Vanilla: ['DET', 'NOUN', 'VERB', 'ADP', 'DET', 'NOUN', '.']
UNK: ['DET', 'NOUN', 'VERB', 'VERB', 'DET', 'NOUN', '.']

```

```

Sentence: ['`', 'The', 'sharp', 'stock', 'market', 'decline', 'in', 'late', 'October', 'appears', '*-1', 'to', 'have', 'had', 'little']
True: ['.', 'DET', 'ADJ', 'NOUN', 'NOUN', 'NOUN', 'ADP', 'ADJ', 'NOUN', 'VERB', 'X', 'PRT', 'VERB', 'VERB', 'ADJ', 'CONJ', 'DET', 'NOUN']
Vanilla: ['.', 'DET', 'ADJ', 'NOUN', 'NOUN', 'NOUN', 'ADP', 'ADJ', 'NOUN', 'VERB', 'X', 'PRT', 'VERB', 'VERB', 'ADJ', 'CONJ', 'DET', 'NOUN']
UNK: ['.', 'DET', 'ADJ', 'NOUN', 'NOUN', 'NOUN', 'ADP', 'ADJ', 'NOUN', 'VERB', 'X', 'PRT', 'VERB', 'VERB', 'ADJ', 'CONJ', 'DET', 'NOUN']

```

Qualitative Analysis

The examples above show cases where the vanilla model incorrectly tagged words while the UNK model produced correct tags.

```
# The UNK model improves performance because it uses tag frequencies when encountering unseen words.
```

```
### Limitations
```

```
# The UNK model does not consider word structure such as suffixes or capitalization. More advanced methods could improve performance fu
```

```
### Conclusion
```

```
# This assignment demonstrated how Hidden Markov Models can be used for Part-of-Speech tagging.
```

```
# The Viterbi algorithm efficiently finds the most probable tag sequence.
```

```
# Handling unknown words significantly improves accuracy.
```